

ASSIGNMENT-1

Python Programming Assignment: Basic Python Concepts

Aim

To apply basic Python programming concepts, specifically **input/output operations**, **arithmetic operations**, and **string concatenation**, by developing two simple programs²²²²²²²²²².

Task to be Done

1. **Task 1: Perform Basic Mathematical Operations** ³
 2. **Task 2: Create a Personalized Greeting** ⁴
-

Requirements

- **Programming Language:** Python
 - **Tools:** A Python development environment (e.g., an IDE or text editor and the Python interpreter).
 - **Submission Platform:** A **GitHub repository**⁵.
 - The repository must contain the Python scripts for **Task 1 and Task 2** (e.g., task1_math.py, task2_greeting.py)⁶.
 - The repository must include a **README.md file** describing the functionality of the programs⁷.
 - The final output should be the **link to the GitHub repository**⁸.
-

Objectives

Task	Objective
Task 1	To demonstrate the ability to take numerical input from a user and perform the four fundamental arithmetic operations (addition, subtraction, multiplication, and division) ⁹ .

Task	Objective
Task 2	To demonstrate the ability to take string input (names) from a user, concatenate them, and use the result in a personalized output string (greeting) ¹⁰ .

Theory

1. User Input in Python

The `input()` function is used to take user input, which is always stored as a **string**. For numerical operations (Task 1), the input string must be converted to a numerical type using functions like `int()` (for integers) or `float()` (for floating-point/decimal numbers)¹¹.

2. Basic Arithmetic Operators (Task 1)

Python supports standard operators for calculations¹²:

- Addition: + ¹³
- Subtraction: - ¹⁴
- Multiplication: * ¹⁵
- Division: / (Performs standard floating-point division) ¹⁶

3. String Concatenation (Task 2)

In Python, the `+` operator is used to join or **concatenate** two or more strings together to form a new single string¹⁷.

Procedure

1. **Write the Python code** for Task 1 as a separate script (e.g., `task1_math.py`).
2. **Test Task 1** with sample inputs (like `$5$` and `$10$`) to ensure the output matches the expected format¹⁸.
3. **Write the Python code** for Task 2 as a separate script (e.g., `task2_greeting.py`).
4. **Test Task 2** with sample inputs (like `"John"` and `"Doe"`) to ensure the output matches the expected format¹⁹.
5. **Create a new GitHub repository.**

6. **Upload** both .py files to the repository.
 7. **Create and upload** a README.md file²⁰.
 8. **Submit** the repository link²¹.
-

Algorithm

Task 1: Basic Mathematical Operations

1. **START**
2. **INPUT:** Prompt the user to enter the **first number** and store it as a numerical variable (num1).
3. **INPUT:** Prompt the user to enter the **second number** and store it as a numerical variable (num2).
4. **PROCESS:** Calculate the **Addition** ($\text{num1} + \text{num2}$).
5. **PROCESS:** Calculate the **Subtraction** ($\text{num1} - \text{num2}$).
6. **PROCESS:** Calculate the **Multiplication** ($\text{num1} * \text{num2}$).
7. **PROCESS:** Calculate the **Division** ($\text{num1} / \text{num2}$).
8. **OUTPUT:** Display the result for **Addition**.
9. **OUTPUT:** Display the result for **Subtraction**.
10. **OUTPUT:** Display the result for **Multiplication**.
11. **OUTPUT:** Display the result for **Division**.
12. **END**

Task 2: Personalized Greeting

1. **START**
2. **INPUT:** Prompt the user to enter their **first name** and store it in a variable (firstName).
3. **INPUT:** Prompt the user to enter their **last name** and store it in a variable (lastName).
4. **PROCESS:** **Concatenate** the first name, a space, and the last name to create the **full name** (fullName).
5. **PROCESS:** Create a **personalized greeting** string that includes the fullName.

6. **OUTPUT:** Display the personalized greeting message.

7. **END**

Code

Task 1: Basic Mathematical Operations (e.g., task1_math.py)

Python

```
# Task 1: Perform Basic Mathematical Operations
```

```
# 1. Take two numbers as input from the user
```

```
# Use float() for potential decimal inputs and standard division
```

```
num1 = float(input("Enter the first number: "))
```

```
num2 = float(input("Enter the second number: "))
```

```
# 2. Perform the basic mathematical operations
```

```
addition = num1 + num2
```

```
subtraction = num1 - num2
```

```
multiplication = num1 * num2
```

```
division = num1 / num2
```

```
# 3. Display the results of each operation
```

```
print(f"\nAddition: {addition}")
```

```
print(f"Subtraction: {subtraction}")
```

```
print(f"Multiplication: {multiplication}")
```

```
print(f"Division: {division}")
```

Task 2: Create a Personalized Greeting (e.g., task2_greeting.py)

Python

Task 2: Create a Personalized Greeting

1. Take a user's first name and last name as input

```
first_name = input("Enter your first name: ")
```

```
last_name = input("Enter your last name: ")
```

2. Concatenate the first name and last name into a full name

Concatenation is done using the '+' operator with a space in between

```
full_name = first_name + " " + last_name
```

3. Print a personalized greeting message using the full name

```
print(f"\nHello, {full_name}! Welcome to the Python program.")
```

Learning Outcomes

Upon completing this assignment, the student will be able to:

- Use the **input()** function to read data from the user²²²²²².
- Convert string inputs to **numerical types** (like float) for arithmetic operations²³.
- Apply basic Python **arithmetic operators** (\$+, -, *, /\$)²⁴.
- Perform **string concatenation** using the + operator²⁵.
- Use the **print()** function effectively to display formatted output, including utilizing f-strings (formatted string literals)²⁶²⁶²⁶²⁶.

RESULT/OUTPUT:

```
Enter the first number: 5
Enter the second number: 10
Addition: 15
Subtraction: -5
Multiplication: 50
Division: 0.5
```

Enter your first name: John

Enter your last name: Doe

Hello, John Doe! Welcome to the Python program.